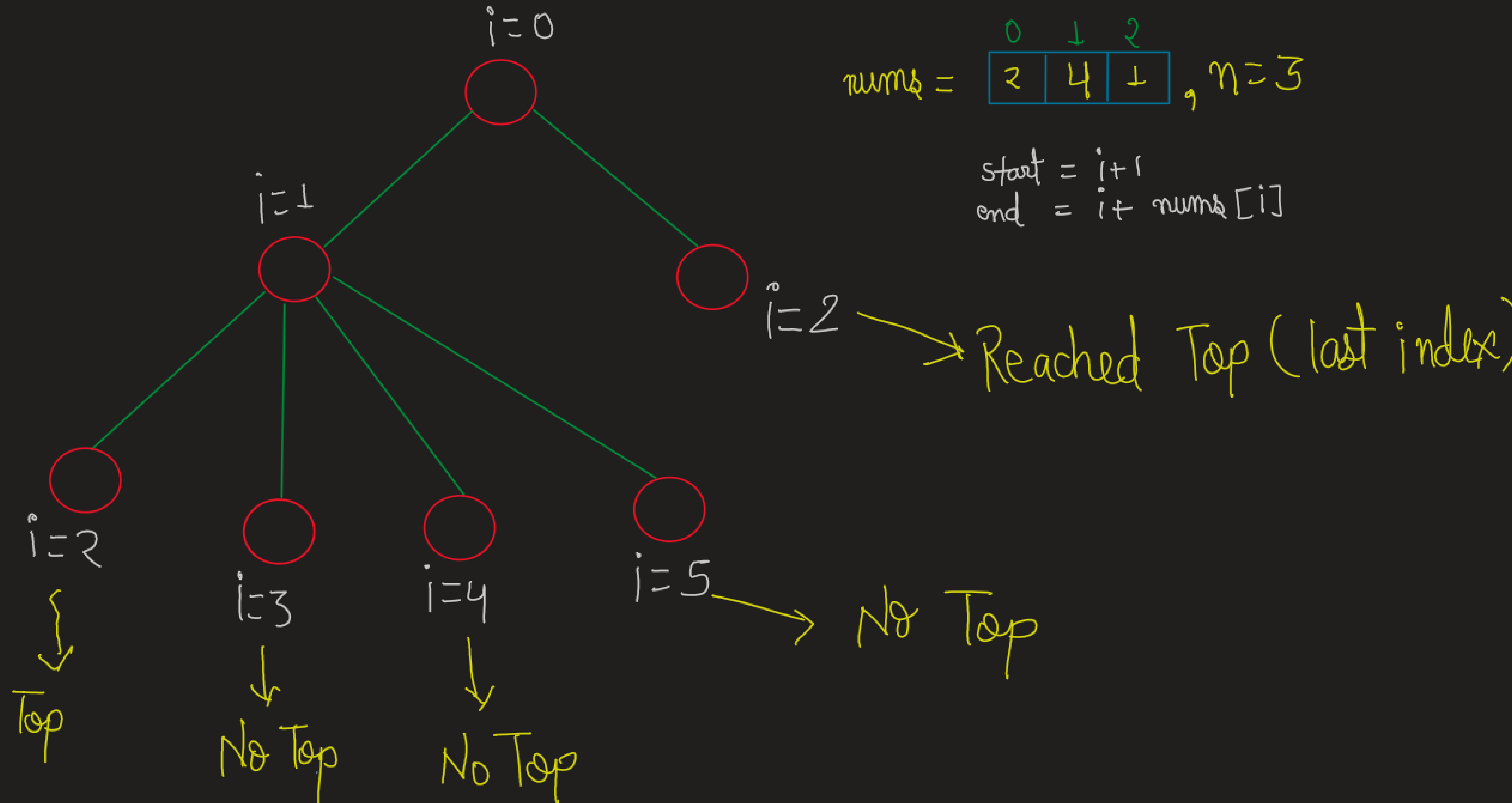
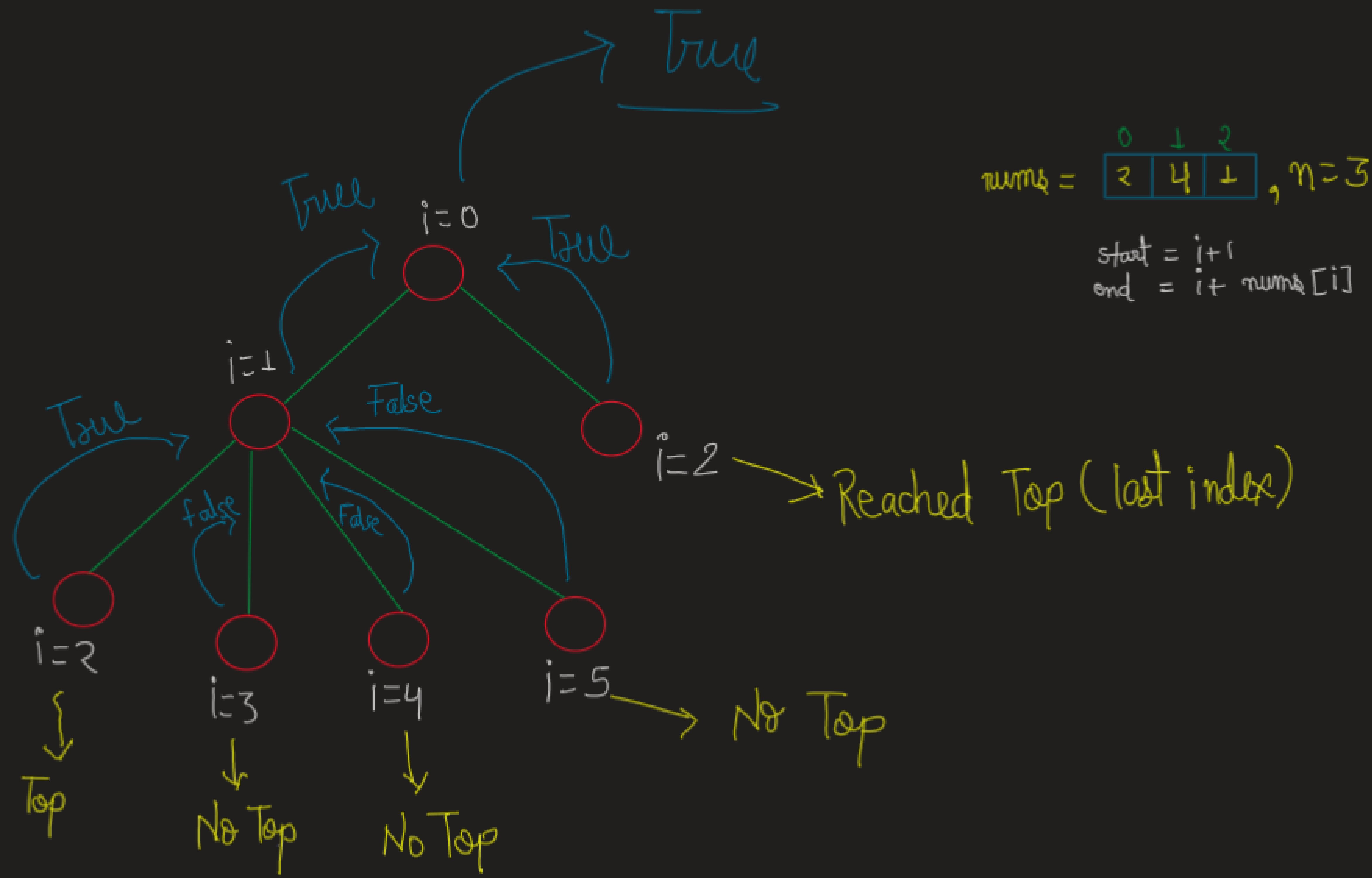
* Tree Diagram:-

$$nums = \begin{array}{ccc} 0 & 1 & 2 \\ \boxed{2} & \boxed{4} & \boxed{1} \end{array}, \; n = 3$$

start = i+1
end = i+ nums[i]





→ If reached top ($i == n-1$) return <u>true</u> as Indication.

→ If ($i >= n$) return <u>false</u> as Indication that we can't reach Top / Last index.

- Base cases are clear now.

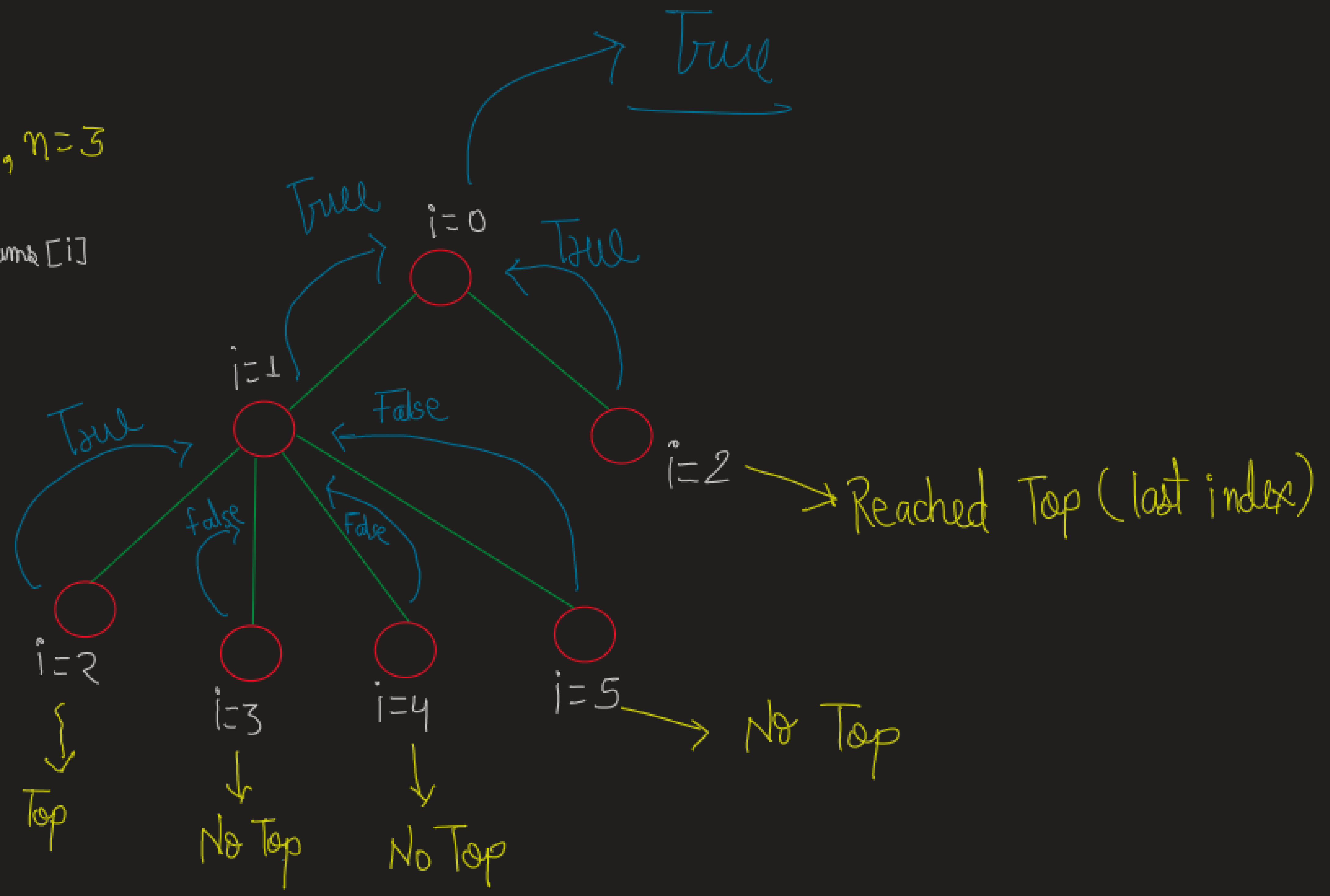
* Recursive Code:-

```
bool Recursion(i, nums) {

    if(i==n-1) return true
    if (i>=n) return false

    int start = i+1
    int end = i+nums[i]

    for(int j=start; j<=end; j++) {
        // Jump from i to jth index
        // we want to find can reach index n-1
        // so call Recursion(J), do the same
        bool canReachEndFrom_j = Recursion(J, nums)
        if (canReachEndFrom_j) return true;
    }

    return false
}
```

	0	1	2
nums =	2	4	1

, n=3

start = i+1
end = i+ nums[i]



* Time → $O(M^N)$ → Exploring from n-1 indices
n-1 = n

→ exploring M options / possibilities on any index.

→ M = maximum value of array

Why It's taking Exponential time?

→ Overlapping Sub problems (Repitative work)
→ Means when n increases, the amount of sub-problem increases
→ solving this repetative problems again and again is increasing time.

→ This concludes that we could optime time in our recursion.
→ Now this hints to DP.
→ so for recursion, we apply Top-Down DP, so lets do Memoization.

→ Now memoize it (Apply DP)

→ let's code it